# SENG 201
# Software Engineering Foundations

[Week 4, Tuesday]



# Types and subtypes

```
private static Note createNote() {...}
private static ContentFragment createFragment() {...}
```

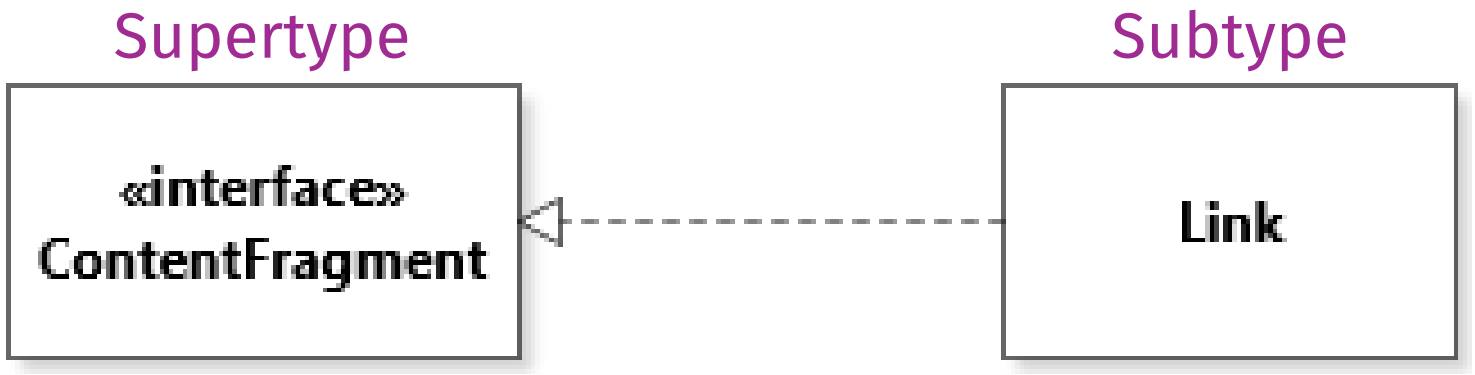
```
Note note = createNote();                          // OK
ContentFragment fragment = createFragment();       // OK

Note note2 = createFragment();                     // ERROR
ContentFragment fragment2 = createNote();          // OK
```

Could be Link


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

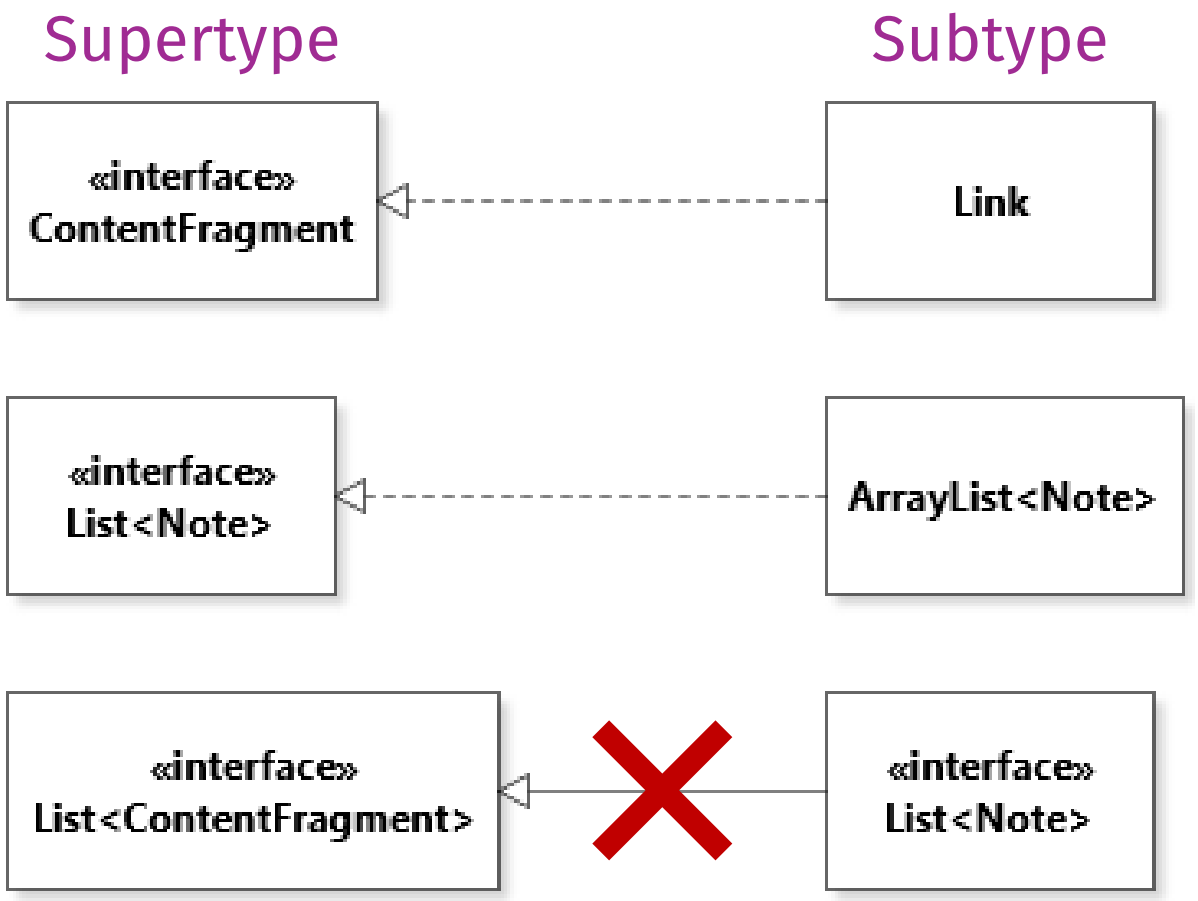
# Types and subtypes




- Declaring an interfaces creates a type
- Declaring a class creates a type
- Implementing an interface creates a subtype


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Types and subtypes




Element type of data structures* must match

*except arrays


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Conversions

We can use assign values of a subtype
to variables of a supertype

```
int small = 0;
long big = small;
```

```
Link link = new Link("https://www.asciiart.eu", "Essential");
ContentFragment copy = link;
```





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Multiple interfaces

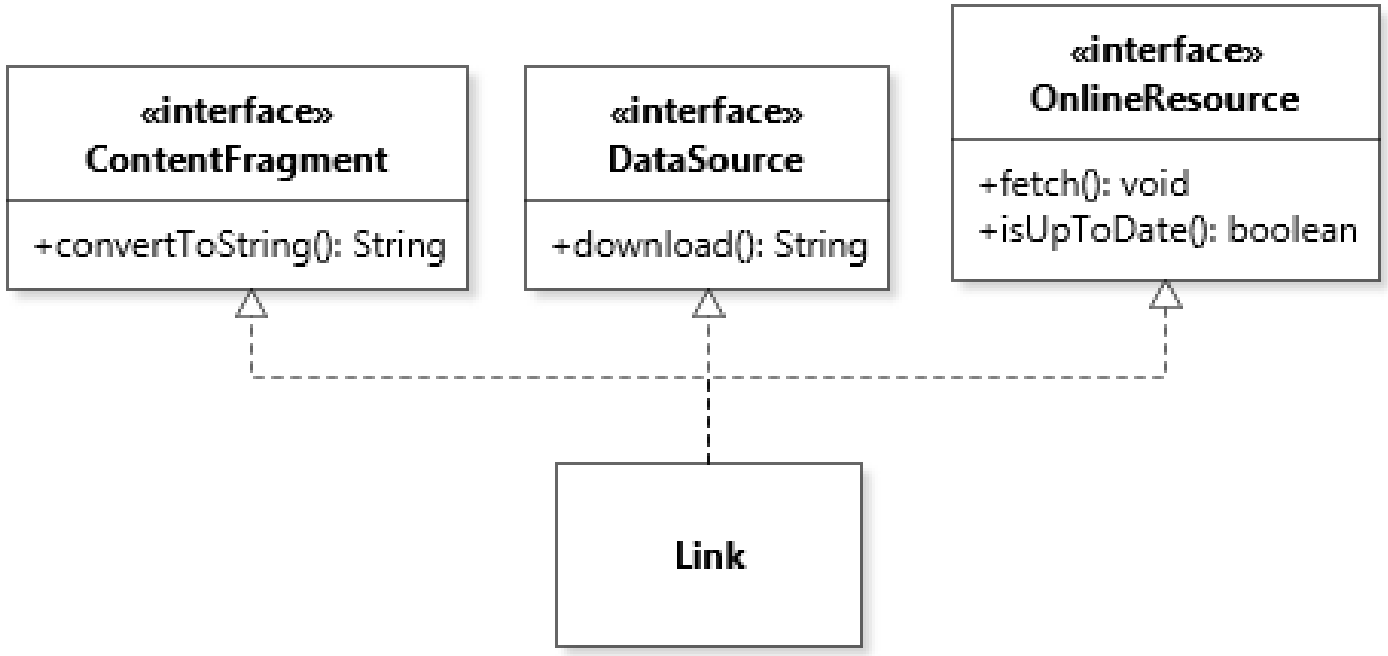
*A class can implement multiple interfaces*

```
public class Link implements ContentFragment, DataSource, OnlineResource {
```

```
@Override
public void fetch() {...}

@Override
public boolean isUpToDate() {...}

@Override
public String download() {...}
```

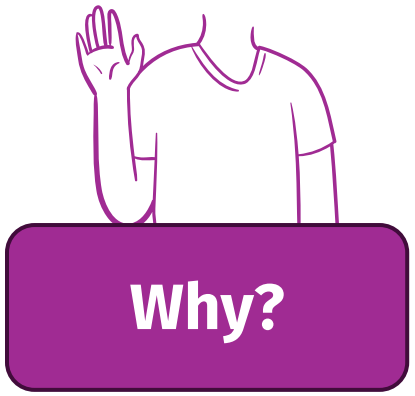


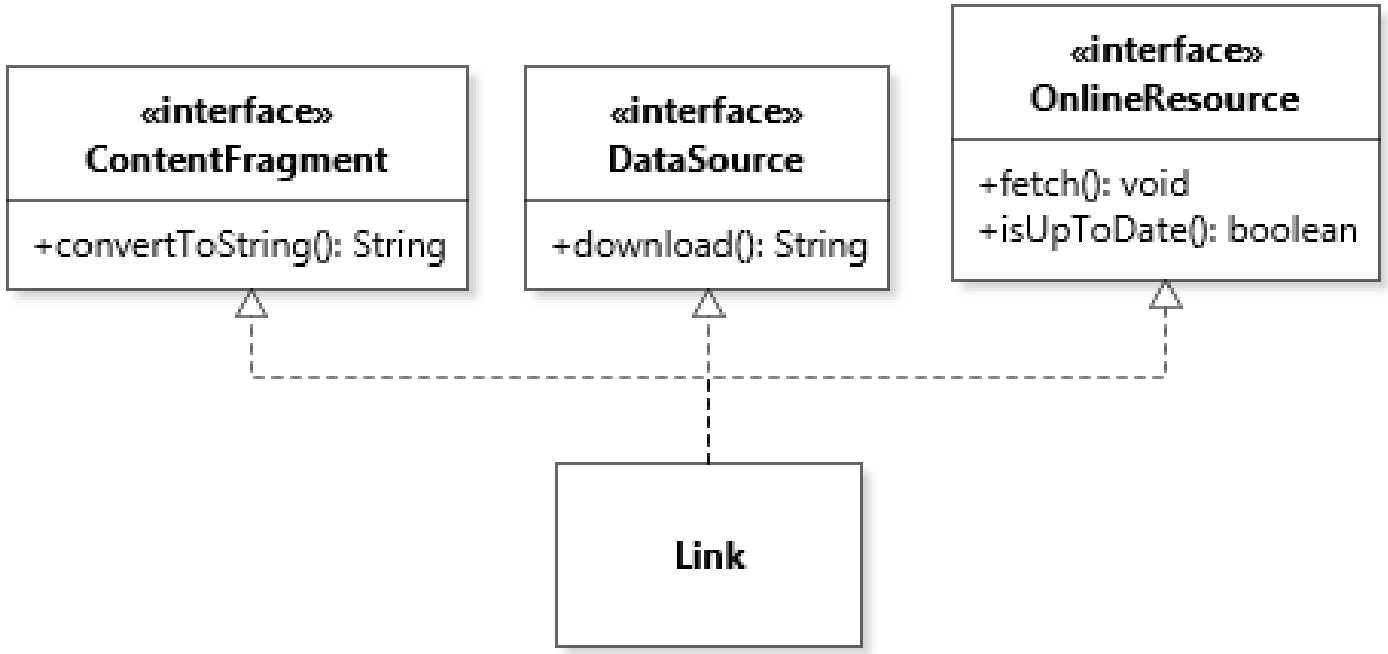


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Multiple interfaces

*A class can implement multiple interfaces*




- Keep interfaces focused and reusable


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

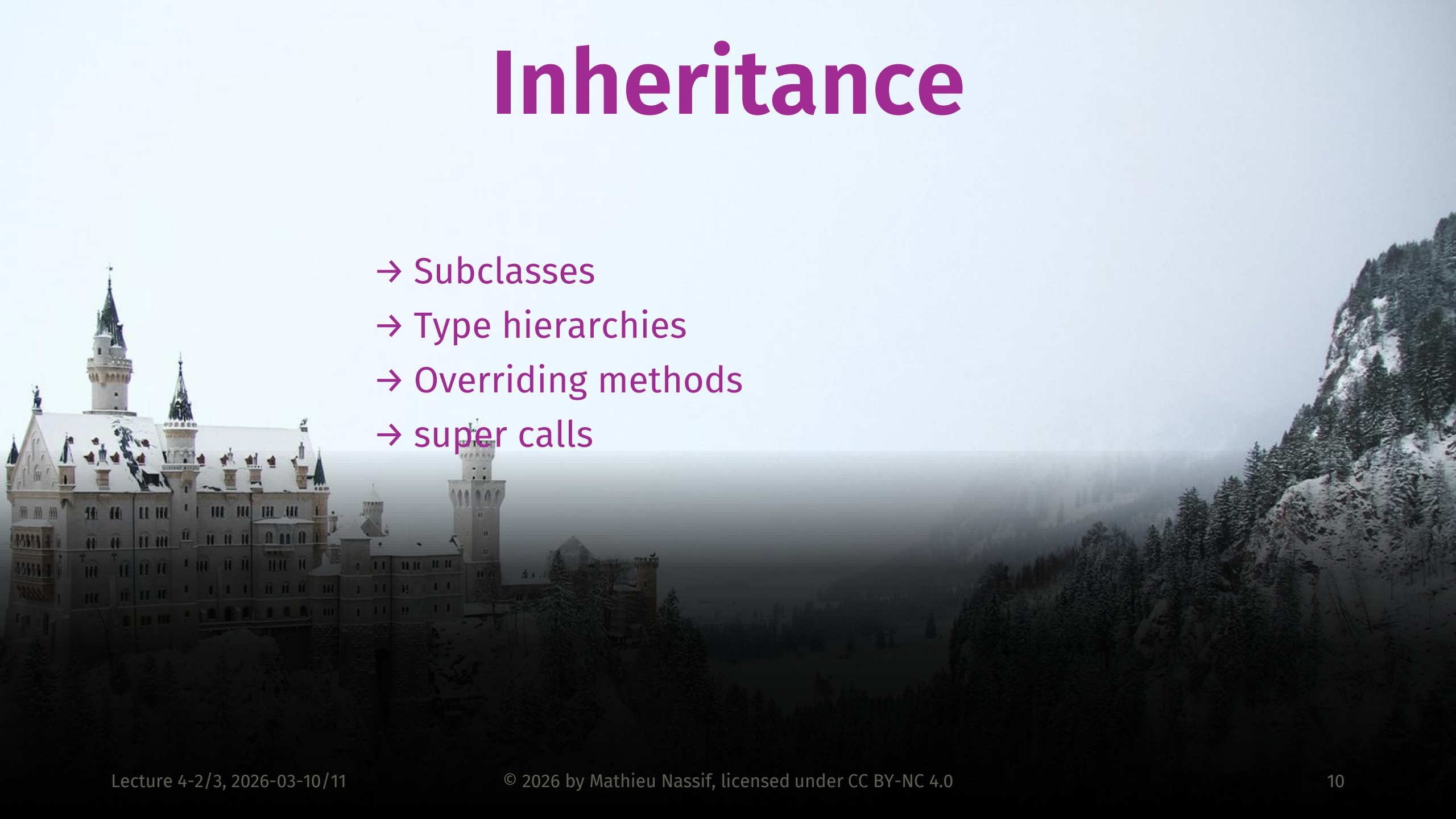
# Multiple interfaces

```
Link link = new Link("https://www.asciiart.eu");
ContentFragment fragment = link;
DataSource source = link;
OnlineResource resource = link;

fragment.convertToString();
source.download();
resource.fetch();
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Inheritance



→ Subclasses

→ Type hierarchies

→ Overriding methods

→ super calls


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

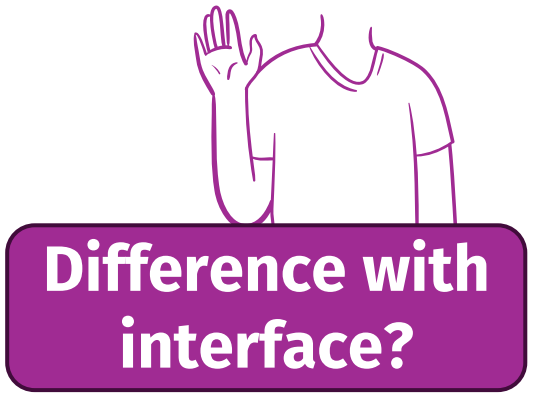
# Creating a subclass

```java
public class Separator implements ContentFragment {

    @Override
    public String convertToString() {
        return "--------------------";
    }
}
```

```java
public class DateSeparator extends Separator {

}
```




- Creating a subclass also creates a subtype

```java
Separator separator = new Separator();
DateSeparator date = new DateSeparator();
Separator alsoDate = date;
date.setSeparatorChar('*');
alsoDate.setSeparatorChar('=');
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Creating a subclass

```java
public class DateSeparator extends Separator {
    // LocalDateTime and Duration are part of java.time package

    public final LocalDateTime date;

    public DateSeparator() {
        date = LocalDateTime.now();
    }

    public long daysSince() {
        return Duration.between(date, LocalDateTime.now()).toDays();
    }
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Creating a subclass

```java
@Override
public String convertToString() {
    return "-------------------- " + date;
}
```

- Overriding a method replaces its original implementation


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Overriding, implementing, overloading

**Implementing:** provide the body of a method for the first time

- Must have the same* signature
- Use @Override keyword to enable compiler checks

**Overriding:** replace the body of a method

- Must have the same* signature
- Use @Override keyword to enable compiler checks

**Overloading:** different methods with the same name

- Must have the same name but different parameter types
- Does not use @Override

*return type can be a subtype and can remove exceptions

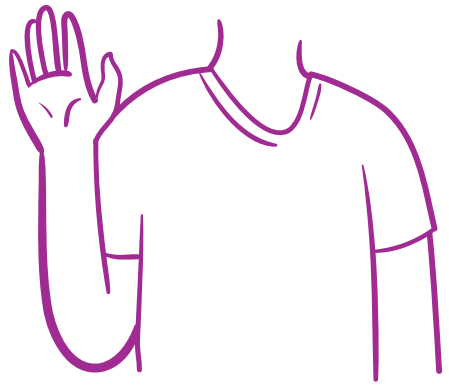

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Overriding, implementing, overloading

ContentFragment.java

```java
public interface ContentFragment {

    public String convertToString();
}
```



Separator.java

```java
public String convertToString() {
    return separator;
}
```

```java
public String convert() {
    return separator;
}
```

```java
public String convertToString(String prefix) {
    return prefix + convertToString();
}
```

DateSeparator.java

```java
public String[] convertToString() {
    return convertToString("Array").split("\\s+");
}
```

```java
public String convertToString() {
    return "-------------------- " + date;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Creating a subclass



What is problematic here?

```java
@Override
public String convertToString() {
    return "-------------------- " + date;
}
```

- Overriding a method replaces its original implementation

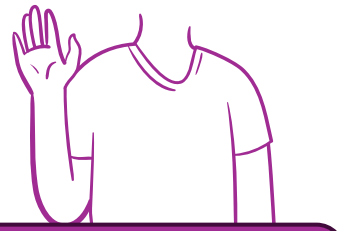

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Accessing superclass members

```
protected String separator = "--------------------";
```

`protected`: Accessible by subclasses (and package)

```
@Override
public String convertToString() {
    return separator + " " + date;
}
```



**What is problematic here?**


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Using methods from superclass

```java
@Override
public String convertToString() {
    return super.convertToString() + " " + date;
}
```

`super`: Force to use parent implementation

`this` refers to this object (dynamic binding)
`super` refers to the parent class (static binding)


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Calling super constructor

```java
public DateSeparator() {
    // implicit super();
    date = LocalDateTime.now();
}

public DateSeparator(char separatorChar) {
    super(separatorChar);
    date = LocalDateTime.now();
}
```

```java
public Separator() {
    this('-');
}

public Separator(char separatorChar) {
    setSeparatorChar(separatorChar);
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Preventing or protecting inheritance

Prevent overriding

```
public final void setSeparatorChar(char c) {
    separator = "";
    for (int i = 0; i < 20; i++) separator += c;
}
```

```
@Override
public void setSeparatorChar(char c) {
    // NO!
}
```

Prevent subclassing

```
public final class DateSeparator
```

```
class NewDateSeparator extends DateSeparator {
```

```
class NewImage extends AsciiImage {
```

```
class NewNote extends Note {
```

Records and enums are always final


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Preventing or protecting inheritance

`final` keyword

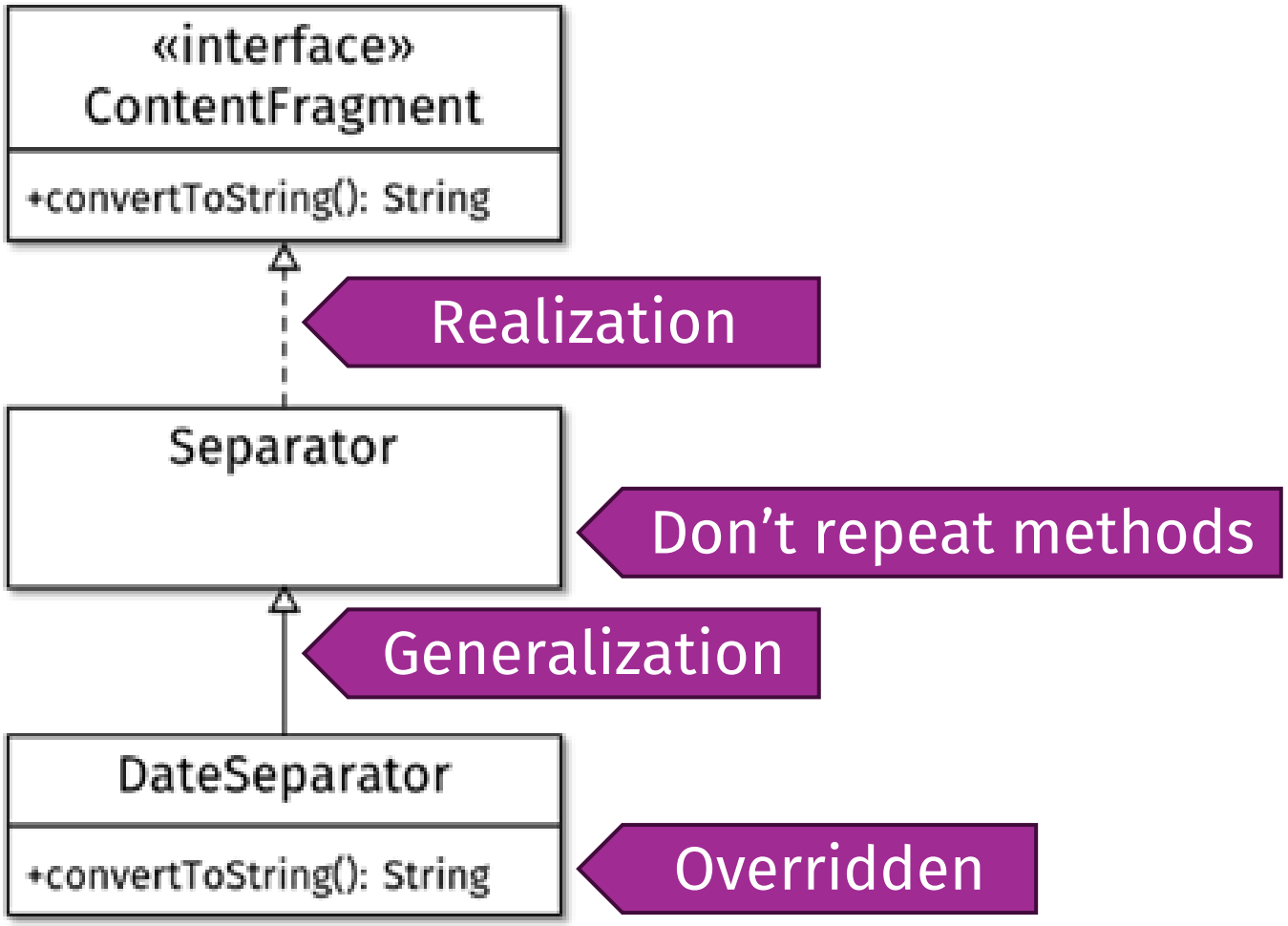
- Variable declaration: prevent reassignment
- Method: prevent overriding
- Class: prevent subclassing


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# UML

DateSeparator is a specialization of Separator.

Separator is a generalization of DateSeparator.





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Single inheritance

*Classes can only inherit (directly) from a single superclass*

```
class Philosopher {
    public String lie() {
        return "This sentence is false.";
    }
}

class Sleeper {
    public String lie() {
        return "zzz";
    }
}
```

```
class SleepingPhilosopher extends Philosopher, Sleeper {}
```

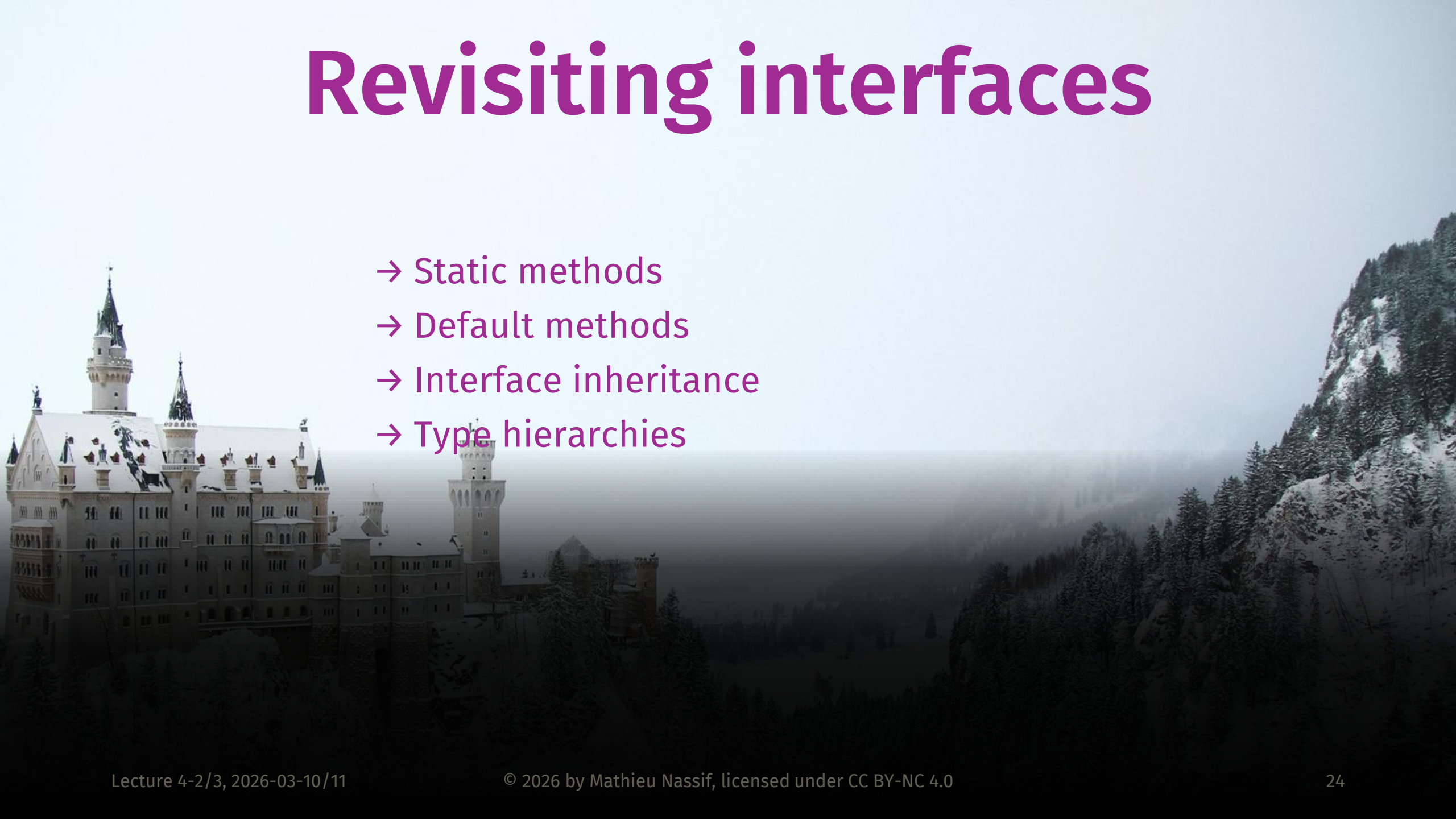
```
SleepingPhilosopher liar = new SleepingPhilosopher();
liar.lie();
```

Which method?

- Classes can implement many interfaces
- Classes can have many subclasses
- But classes can only have one parent class


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Revisiting interfaces



→ Static methods

→ Default methods

→ Interface inheritance

→ Type hierarchies


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Static interface methods

```java
public static final char DEFAULT_FRAME_CHAR = '-';

public static char getDefaultFrameChar() {
    return DEFAULT_FRAME_CHAR;
}
```

Must be a constant (public static final)

Must be public or private



**Overriding or overloading?**

```java
public Separator() {
    this(ContentFragment.getDefaultFrameChar());
}
```

```java
public class Separator implements ContentFragment {

    public static char getDefaultFrameChar() {
        return '*';
    }
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Default interface methods

ContentFragment.java

```java
public default int getDefaultLineLength() {
    return 20;
}
```

DateSeparator.java

```java
@Override
public int getDefaultLineLength() {
    return 10;
}
```

Separator.java

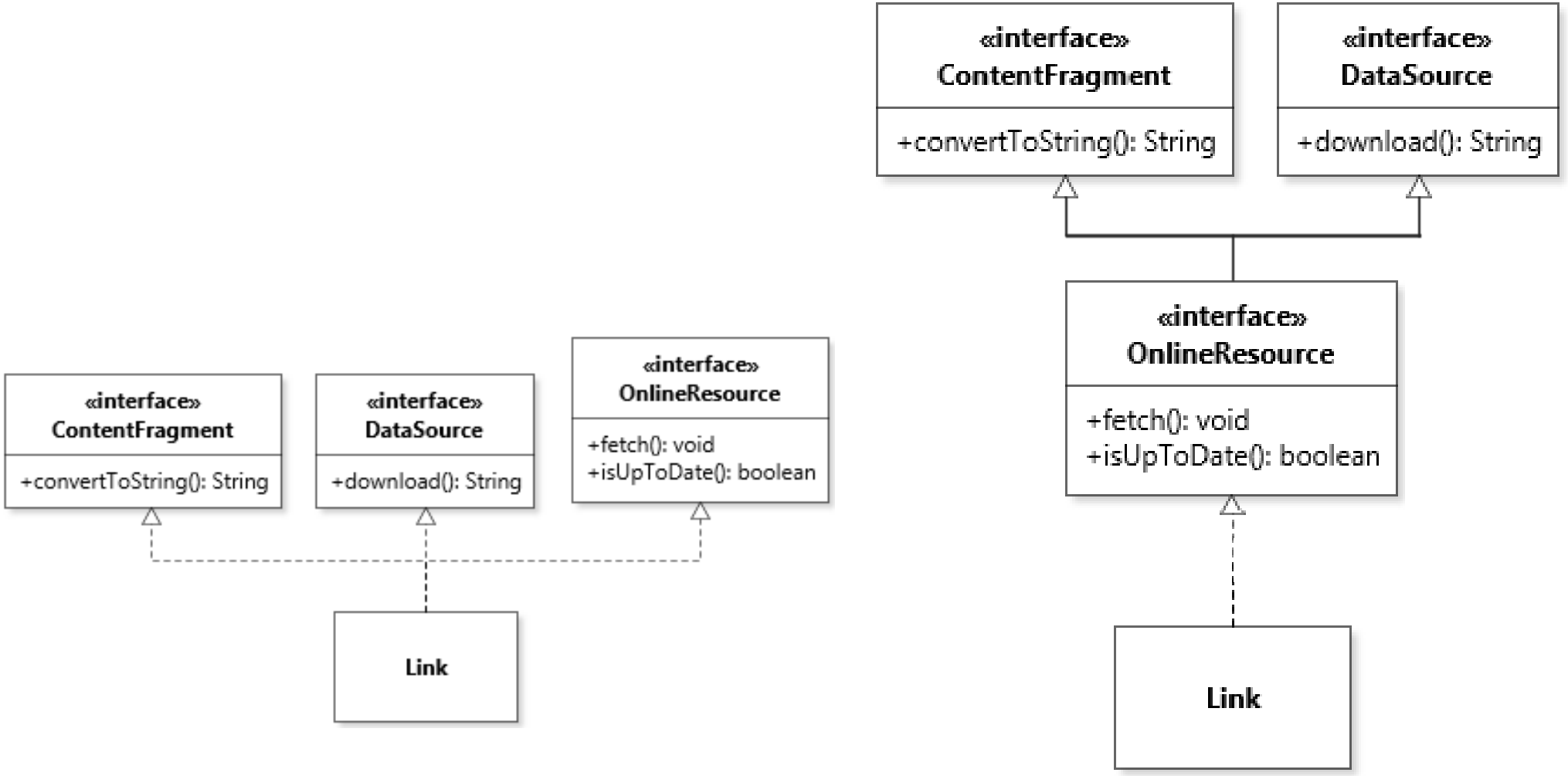
```java
public final void setSeparatorChar(char c) {
    separator = "";
    int length = getDefaultLineLength();
    for (int i = 0; i < length; i++) separator += c;
}
```




Dual inheritance…


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Interface inheritance

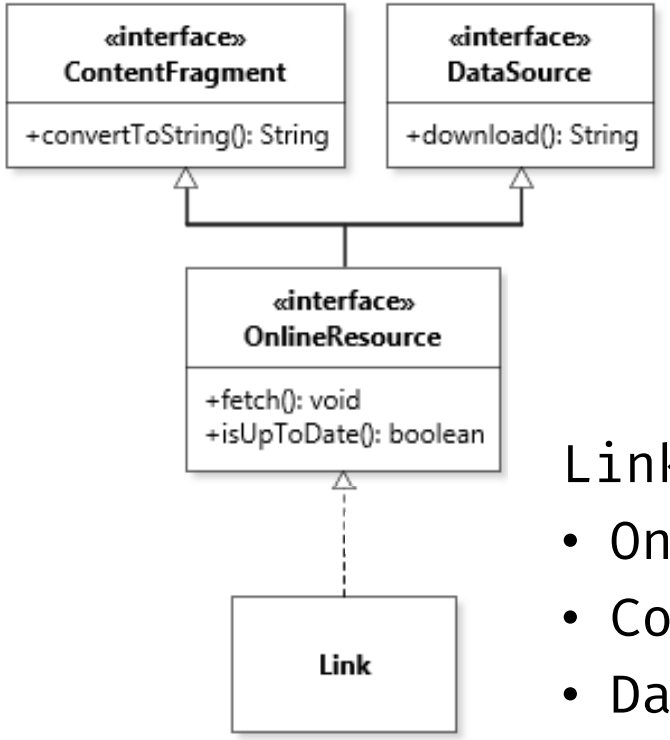




© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Type hierarchies

*The subtype relationship is transitive*




```
Link link = new Link("https://www.asciiart.eu/animals/bears");
OnlineResource resource = link;
ContentFragment fragment = link;
DataSource source = link;

ContentFragment fragment2 = resource;
DataSource source2 = resource;
```

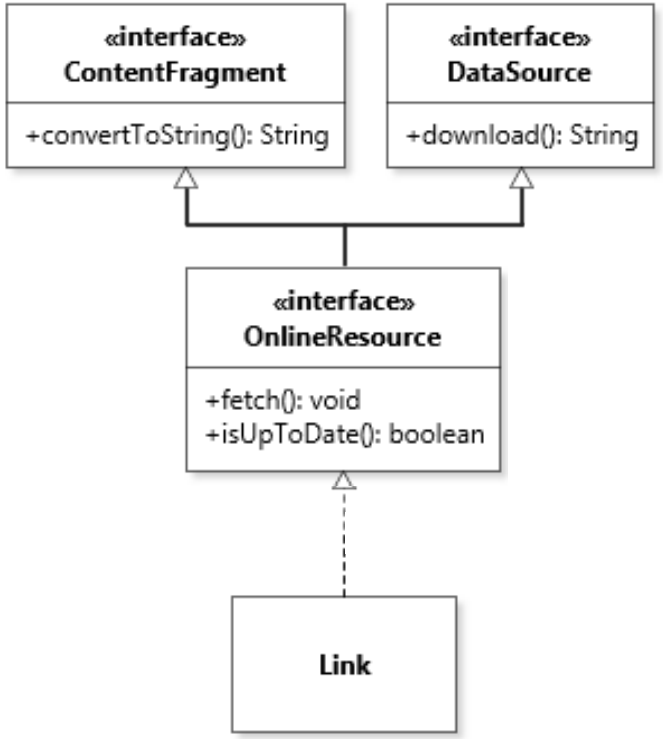
`Link` is still a subtype of

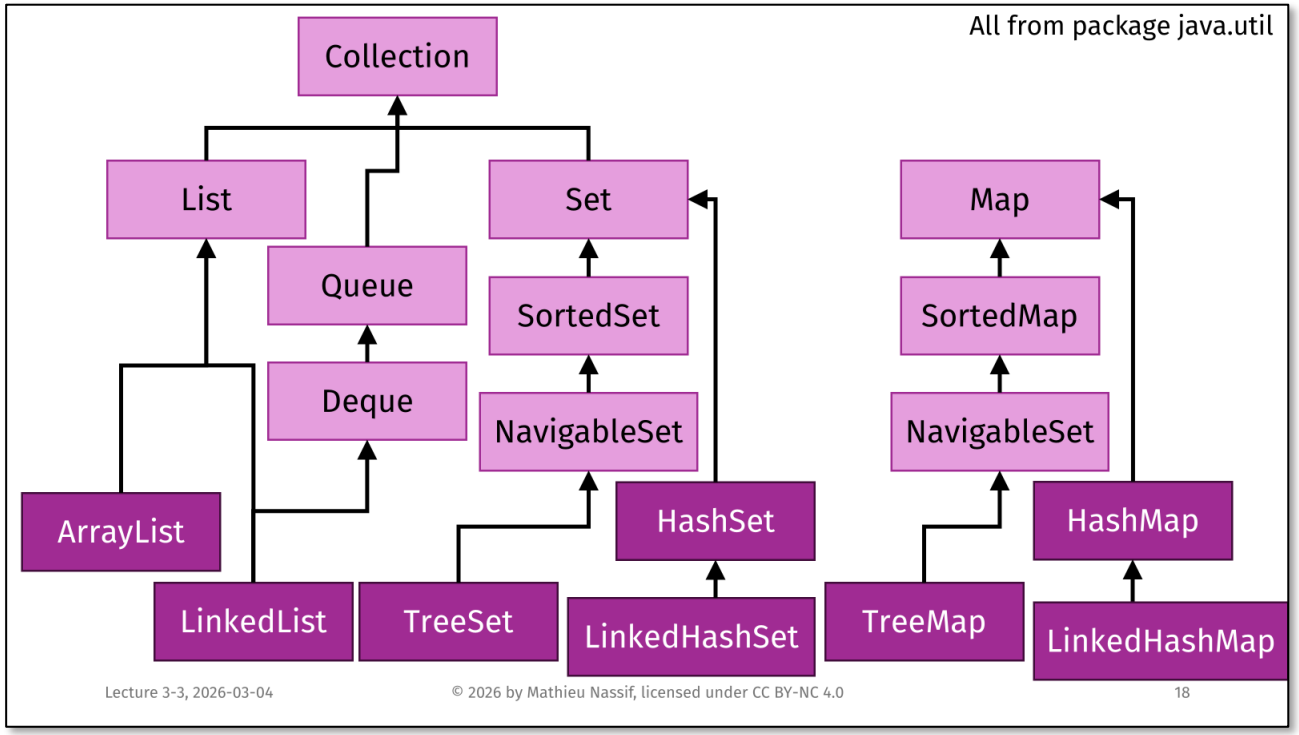
- `OnlineResource`,
- `ContentFragment`, and
- `DataSource`


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Type hierarchies

*The subtype relationship is transitive*





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Type hierarchies

*The subtype relationship is transitive*

Categorize abstractions (classes, interfaces) into hierarchies

Dual meaning of inheritance

- Subtype: “is a” relationship

  *a DateSeparator “is a” Separator*

- Extension: inherit behaviour from parent and expands on it

  *a DateSeparator reuse methods from Separator and does more*


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Substitutability

Liskov Substitution Principle (LSP)

*"Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program."*

Not just about syntax!

Overridden methods should not

- have more constraints
- have lesser guarantees


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Substitutability

```
public class MessageSeparator extends Separator {

    private String openingSequence;
    private String closingSequence;
```

```
public void setOpenCloseSequences(String open, String close) {
    openingSequence = open;
    closingSequence = close;
}
```

```
@Override
public String convertToString() {
    if (openingSequence == null || closingSequence == null) {
        System.out.println("ERROR: missing opening or closing sequence");
        return null;
    }
    return openingSequence + message + closingSequence;
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Substitutability

```
Separator separator = new Separator();
System.out.println(separator.convertToString()); // OK

Separator date = new DateSeparator();
System.out.println(date.convertToString()); // OK

Separator message = new MessageSeparator("Bad LSP");
System.out.println(message.convertToString()); // PROBLEM!
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Explicit casting

Explicit conversion if we know the subtype

```
ContentFragment fragment = new DateSeparator();
Separator downcast = (Separator) fragment;
DateSeparator furtherDown = (DateSeparator) fragment;
```

Use `instanceof` to check subtypes

```
Separator unknown = createSeparator();
if (unknown instanceof MessageSeparator) {
    MessageSeparator msg = (MessageSeparator) unknown;
    msg.setOpenCloseSequences("<<<", ">>>");
}
System.out.println(unknown.convertToString());
```

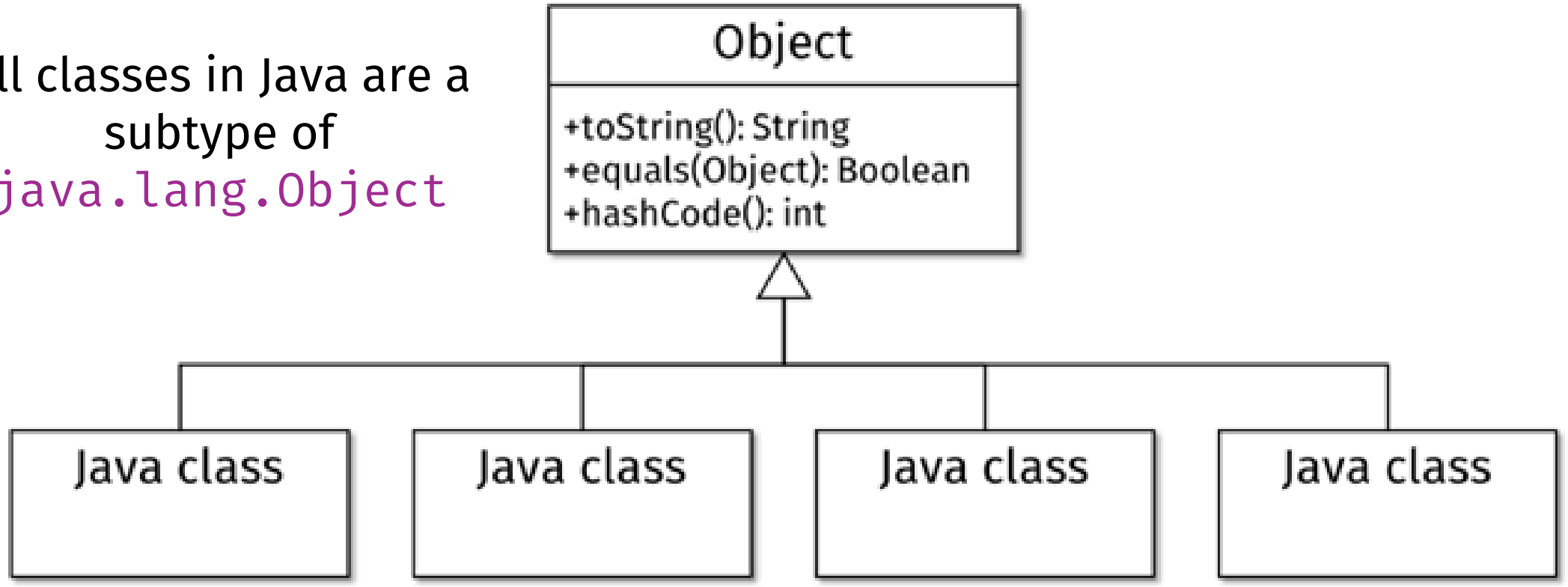
Can be a sign of poor design

Look for pattern matching in Java for more advanced syntax


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Object class

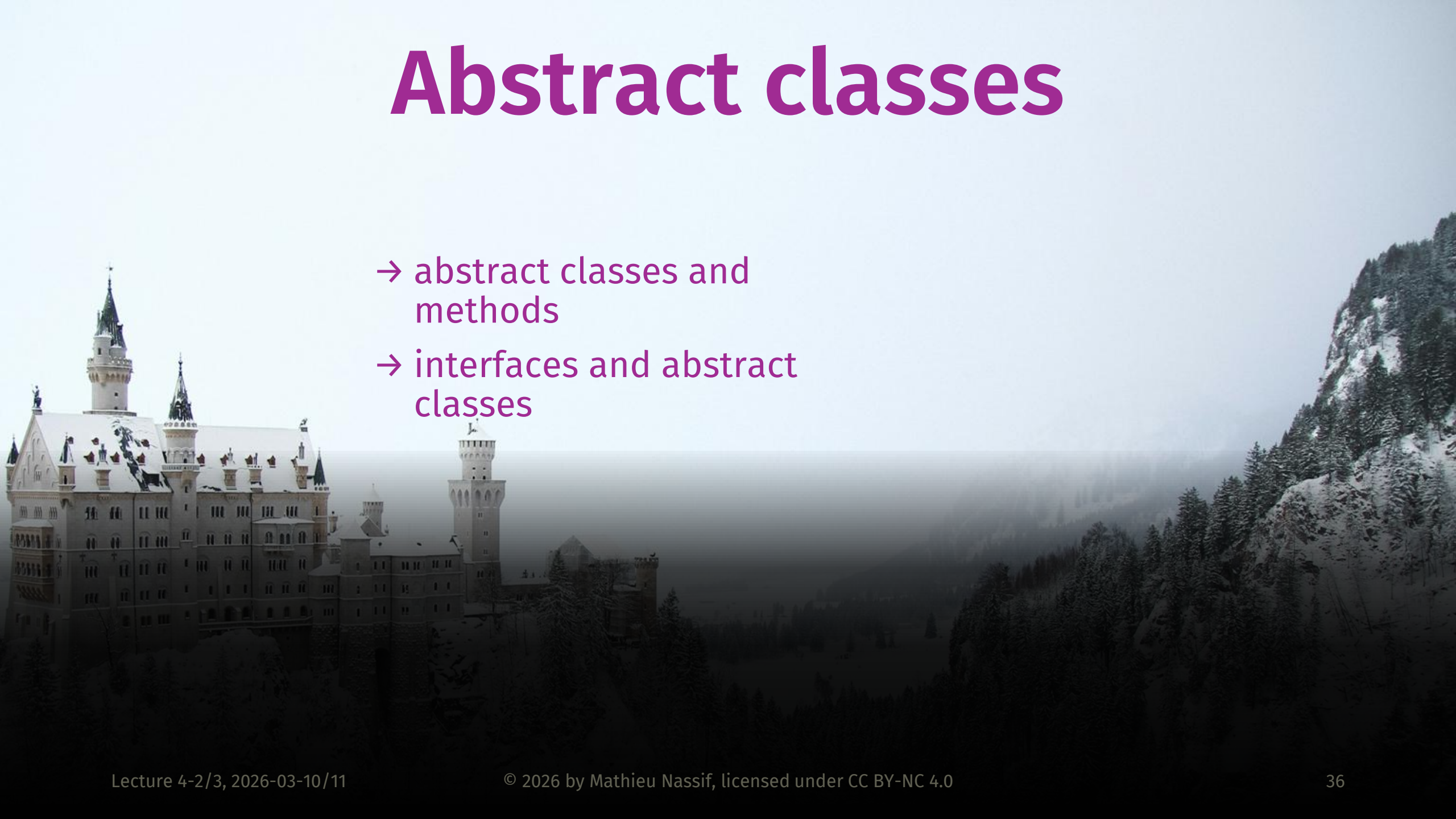
All classes in Java are a subtype of `java.lang.Object`





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Abstract classes



- → abstract classes and methods
- → interfaces and abstract classes


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# “Incomplete” implementation

- Inheritance is a technique for code reuse
- What if we have a only partial implementation that we want to reuse?


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Abstract classes

```java
public abstract class AbstractSeparator implements ContentFragment {

    private String separator;

    public AbstractSeparator(char separatorChar) {
        setSeparatorChar(separatorChar);
    }

    @Override
    public String convertToString() {
        return separator;
    }

    public final void setSeparatorChar(char c) {
        separator = "";
        int length = getDefaultLineLength();
        for (int i = 0; i < length; i++) separator += c;
    }
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Abstract classes

```java
public final void setSeparatorChar(char c) {
    separator = "";
    int length = getSeparatorLength();
    for (int i = 0; i < length; i++) separator += c;
}

protected abstract int getSeparatorLength();
```

Separator.java

```java
@Override
protected int getSeparatorLength() {
    return getDefaultLineLength();
}
```

DateSeparator.java

```java
@Override
protected int getSeparatorLength() {
    return Math.min(5,
            getDefaultLineLength() - date.toString().length());
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Abstract classes

- Cannot be instantiated

```
new AbstractSeparator('-')
```

is illegal even if the constructor exists and is public.

- Can do anything else a class can do
  - Define constructors
  - Have fields, methods
  - Implement interfaces, extend a class, be extended

- Can have abstract methods


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# Abstract classes vs interfaces

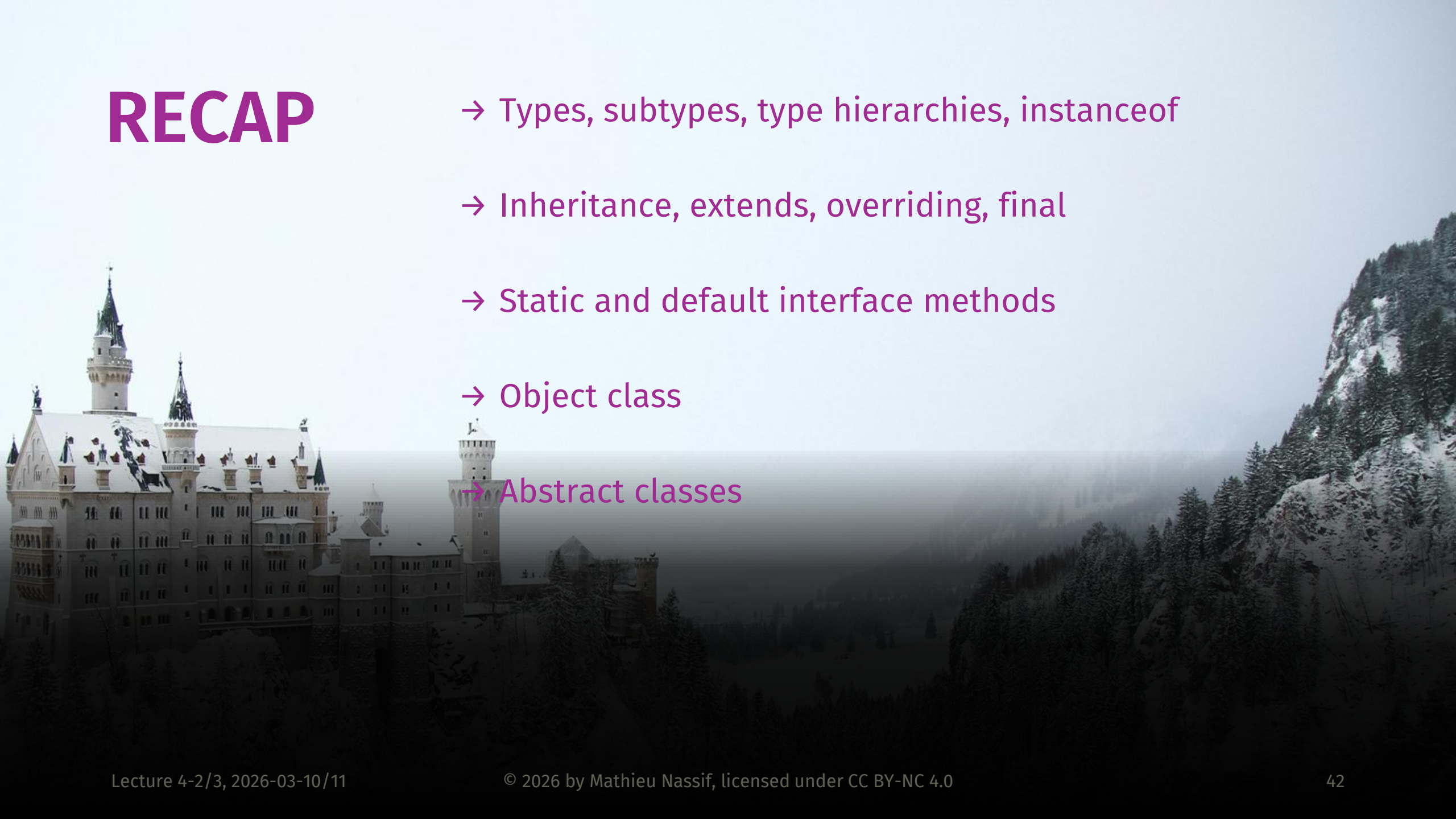
- Abstract classes represent a partial implementation
- Interfaces represent… an interface

- Abstract classes can technically emulate
  - interfaces (have only abstract methods)
  - normal classes (have no abstract methods)

- It’s about the intention, not just the code


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

# RECAP



→ Types, subtypes, type hierarchies, instanceof

→ Inheritance, extends, overriding, final

→ Static and default interface methods

→ Object class

→ Abstract classes


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0